I4.0 Holonic Multi-agent Testbed Enabling Shared Production

Alexis T. Bernhard, Simon Jungbluth, Ali Karnoub, Aleksandr Sidorenko, William Motsch, Achim Wagner, and Martin Ruskowski

1 Introduction

Nowadays, globalization changes the manufacturing environment significantly. Global business signifies global competition and intends a need for shorter product life cycles, whereas consumer-oriented businesses foster customized products. However, rigidly connected supply chains have proven vulnerable to disruptions. Therefore, requirements are changing focusing on adaptability, agility, responsiveness, robustness, flexibility, reconfigurability, dynamic optimization and openness to innovations. Isolated and proprietary manufacturing systems need to move ahead with decentralized, distributed, and networked manufacturing system architectures [20]. Cloud Manufacturing could be a solution for highly diversified and reconfigurable supply chains. Liu et al. define it as "[. . .] a model for enabling the aggregation of distributed manufacturing resources [. . .] and ubiquitous, convenient, on-demand network access to a shared pool of configurable manufacturing services

A. T. Bernhard (✉) · A. Sidorenko · W. Motsch · A. Wagner
Deutsches Forschungszentrum für Künstliche Intelligenz GmbH (DFKI), Kaiserslautern, Germany
e-mail: alexis.bernhard@dfki.de; aleksandr.sidorenko@dfki.de; william.motsch@dfki.de; achim.wagner@dfki.de

S. Jungbluth · A. Karnoub
Technologie-Initiative SmartFactory KL e.V., Kaiserslautern, Germany
e-mail: simon.jungbluth@smartfactory.de; ali.karnoub@smartfactory.de

M. Ruskowski
Deutsches Forschungszentrum für Künstliche Intelligenz GmbH (DFKI), Kaiserslautern, Germany

Technologie-Initiative SmartFactory KL e.V., Kaiserslautern, Germany
e-mail: martin.ruskowski@smartfactory.de; martin.ruskowski@dfki.de

© The Author(s) 2024

J. Soldatos (ed.), *Artificial Intelligence in Manufacturing*,
https://doi.org/10.1007/978-3-031-46452-2_13

that can be rapidly provisioned and released with minimal management effort or interaction between service operators and providers" [22].

SmartFactory^{KL} shares their vision with the terminology *Production Level 4* and Shared Production (SP) [2] that represents reconfigurable supply chains in a federation of trusted partners that dynamically share manufacturing services over a product lifecycle through standardized Digital Twins. Hence, it represents an extension of Cloud Manufacturing.

This vision requires sharing data on products and resources. Production data contains sensitive information, leading to fear of data misuse. Thus, additionally to interoperability and standardization, data sovereignty is one major aspect when building a manufacturing ecosystem. In [15], a technology-dependent solution is presented. The authors propose using Asset Administration Shells (AAS) and the concepts of Gaia-X to exchange data in a self-sovereign interoperable way. In this sense, AAS aims to implement a vendor-independent Digital Twin [10], while Gaia-X creates a data infrastructure meeting the highest standards in terms of digital sovereignty to make data and services available. Key elements are compliance with European Values regarding European Data Protection, transparency and trustability [3]. Nevertheless, the authors of [15] focus more on the business layer and the data exchange process rather than intelligent decision-making processes. Due to their characteristics, Multi-Agent System (MAS) seems to be a promising natural choice for implementing the logic and interactions among different entities.

In our contribution, we present the structure of MAS for a modern production system to cope with the upper mentioned challenges. We focus on the intra-organizational resilience of the shop-floor layer to provide the necessary flexibility to enable an SP scenario. In Sect. 2, the considered concepts for the usage of MAS in industrial applications are described. Section 3 describes the architecture of the applied MAS and Sect. 4 gives an overview about the characteristics of a plant specific holonic MAS and implements a prototype in the demonstrator testbed at *SmartFactory*^{KL}.

2 State of the Art

To use MAS in modern production environments, MAS needs to manage the complexity of fractal control structures of the shop-floor layer. Therefore, we elaborate on the control structures of industrial manufacturing systems (see Sect. 2.1). In Sect. 2.2, terminology like skill-based approach and cyber-physical production systems (CPPS) are mentioned to deal with the encapsulation of production environment and separate the implementation from the functionality. Section 2.3 compares Agents and Holons and relates the upper mentioned concepts with the holonic paradigm.

2.1 Control Architectures in the Manufacturing Domain

Recently, computerization of industrial machines and tools led to hardware equipped with some kind of software-controlled intelligence. Digitalization trends enable and empower flexibility, shifting static to dynamic optimization. Contributions like [15, 23, 30] list the need for common information models, standardized interfaces, real-time and non-real-time cross-layer communication, separation of concern, flexibility, semantics, intelligence, scalability, inter-enterprise data exchange, collaborative style of works, privacy and self-sovereignty.

Leitão and Karnouskos [20] identify three principal types of control structures in industrial manufacturing systems: centralized, (modified) hierarchical and (semi-)heterarchical architectures. A *centralized* architecture has only one decision-making entity at the root of the system. It handles all planning and control issues, and the other entities have no decision power. The centralized architecture works best in small systems where short paths lead to effective optimization in a short amount of time. A *hierarchical* organization of control distributes the decision-making overall hierarchical levels. Higher levels make strategic-oriented decisions, and lower levels focus on simple tasks. Such architectures can be efficient in static production environments and are more robust to failures than centralized control structures. The hierarchical architectures are typical for the computer-integrated manufacturing paradigm. In contrast to the master-slave flow of control in the hierarchical structures, the *heterarchical* architectures rely on cooperation and collaboration for decision-making. In fully heterarchical architectures, there are no hierarchies at all and each entity is simultaneously master and slave. Such an organization is typical for default MASs. These control structures are highly flexible, but the global optimization goals are hard to achieve because knowledge and decisions are localized by each agent and require a number of interactions between the agents to make them global. This was one of the major criticism of classical MAS architectures. This led to the invention of semi-heterarchical control structures, which are also called loose or flexible hierarchies. Lower levels of such architectures should react quickly to disturbances and make fast decisions. These levels are characterized by hierarchical organization and mostly reactive agents. The higher levels appreciate the flexibility of heterarchical structures and intelligent decision-making by deliberative agents. Semi-heterarchical control structures are typical for so-called holonic architectures.

2.2 Cyber-Physical Production Systems

A flexible and modular production environment manifests in the concept of Cyber-Physical Production Modules (CPPMs), which provide standardized interfaces to offer different functionalities as a service [17]. Therefore, the skill-based approach aims to encapsulate the production module's functionalities and decouples them

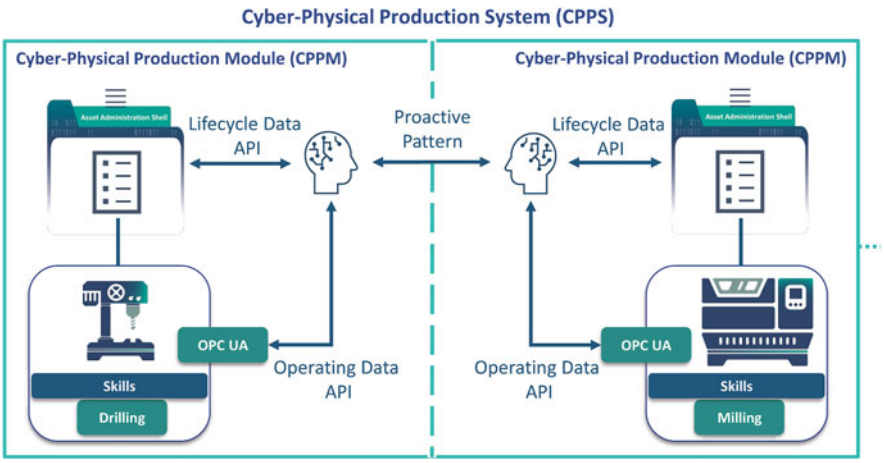

Fig. 1 Representation of a cyber-physical production system

from the specific implementation with the aim of increasing the flexibility. As visualized in Fig. 1, CPPMs can be combined into a CPPS to perform control tasks and react to information independently. CPPS connects the physical and digital worlds and react to information from the environment and environmental influences [8].

CPPMs require a need for a vendor-independent self-description to perform production planning and control. AAS represents an approach to achieve this standardized digital representation, where submodels (SM) are used to describe domain-specific knowledge [10]. The *Plattform Industrie 4.0* proposes an information model composing the concepts of capabilities, skills and services as machine-readable description of manufacturing functions to foster adaptive production of mass-customizable products, product variability, decreasing batch size, and planning efficiency. The Capability can be seen as an "implementation-independent specification of a function [. . .] to achieve an effect in the physical or virtual world" [25]. Capabilities are meant to be implemented as skills and offered as services in a broader supply chain. From the business layer, the service represents a "description of the commercial aspects and means of provision of offered capabilities" [25]. From the control layer, *Plattform Industrie 4.0* defines a Production Skill as "executable implementation of an encapsulated (automation) function specified by a capability" [25] that provides standardized interfaces and the means for parametrization to support their composition and reuse in a wide range of scenarios. The skill interface is mostly realized with OPC UA, since it has proven itself in automation technology. Topics such as OPC UA and AAS can lead to confusion regarding the separation of concerns. AAS is used as linkage to the connected world and lifecycle management in adherence to yellow pages, whereas OPC UA is applied for operative usage.

2.3 Agents and Holons

The study of MAS began within the field of Distributed Artificial Intelligence. It investigates the global behavior based on the agent's fixed behavior. The studies compromise coordination and distribution of knowledge. In this context, Leitão and Karnouskos define an agent as "[. . .] an autonomous, problem-solving, and goal-driven computational entity with social abilities that is capable of effective, maybe even proactive, behavior in an open and dynamic environment in the sense that it is observing and acting upon it in order to achieve its goals" [20]. MAS is a federation of (semi-)autonomous problem solvers that cooperate to achieve their individual, as well as global system's goals. To succeed, they rely on communication, collaboration, negotiation, and responsibility delegation [20]. MAS was motivated by subjects like autonomy and cooperation as a general software technology, while the emergence in the manufacturing domain has been growing recently.

The holonic concept was proposed by Koestler to describe natural beings that consist of semi-autonomous sub-wholes that are interconnected to form a whole [16]. Holonic Manufacturing System (HMS) is a manufacturing paradigm proposed at the beginning of the 1990s as an attempt to improve the ability of manufacturing systems to deal with the evolution of products and make them more adaptable to abnormal operating conditions [7]. Holonic production systems are fundamentally described in the reference architecture PROSA, with the aim of providing production systems with greater flexibility and reconfigurability [32]. A Holon is an autonomous, intelligent, and cooperative building block of a manufacturing system for transformation, transportation, storing and / or validating information and physical objects [33]. As shown in Fig. 2, a Manufacturing Holon always has an information processing part and often a physical processing part [4]. Holons join holarchies that define the rules for interaction between them. Each Holon can be simultaneously a part of several holarchies and as well as a holarchy itself. This enables very complex and flexible control structures, also called flexible hierarchies. It is important to note that the cooperation process also involves humans, who might enter or exit the Holon's context [4]. In summary, HMS can be seen as an analogy to CPPS, where skills provide the control interface to the physical processing parts.

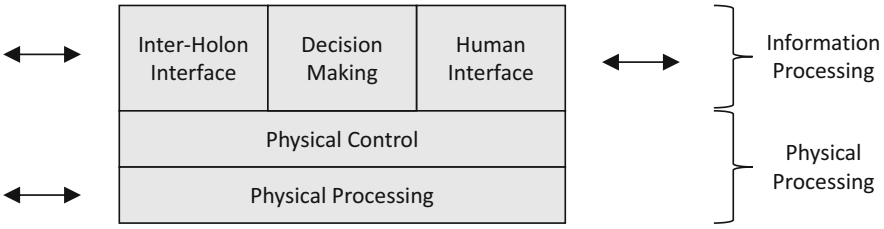

Fig. 2 General Architecture of a Holon in accordance to [7]

Previous research investigated the terminology between Agents and Holons. Giretti and Botti [7] perform a comparative study between the concepts of Agents and Holons. The authors explain that both approaches differ mainly in motivation but are very closely related, and a Holon can be treated as a special type of agent with the property of recursiveness and connection to the hardware. Subsequently, the authors define a recursive agent. One form of a MAS can be a holonic MAS, where there is a need for negotiation to optimize the utility of individual agents as well as the utility of their Holons [1]. The choice is more determined by the point of view. On the other hand, Valckenaers [31] explains that HMS and MAS frequently are seen as similar concepts, but it is important to be aware of contrasting views. In the author's opinion, it no longer holds to see MAS as a manner to implement HMS. HMS researches' key achievement is the absence of goal-seeking. The authors explicit distinguish between intelligent beings (Holons) to shadow the real-world counterpart and intelligent agents as decision makers. We want to sharpen the view and raise awareness of the different wording. Nevertheless, we treat a Holon as a special type of Agent.

The ADACOR architecture presents an example of a holonic MAS that provides a multi-layer approach for a distributed production and balancing between centralized and decentralized structures to combine global production optimization with flexible responses to disturbances [19].

Regarding modular production systems, the concept of Production Skills of CPPMs is important for usage in a multi-agent architecture to interact with a hardware layer [28]. To interact in a dynamic environment, agents furthermore need an environment model to describe the agent's knowledge and a standardized communication. The former serves to describe the respective domain of the agent, to configure the agent's behavior and aggregate information from external knowledge basis or other agents [20]. It requires a standardized information model to ensure autonomous access. In adherence to CPPS, AAS is suitable to describe the agents' properties, i.e., communication channels, physical connection, identification and topology. The latter might be direct or indirect. Direct communication means an exchange of messages. Known communication languages like Knowledge Query and Manipulation Language (KQML) or Agent Communication Language (ACL) rely on speech acts that are defined by a set of performative (agree, propose). Indirect communication relies on pheromone trails of ants, swarm intelligence, the concept of a blackboard, or auctions [20]. Interactions between AAS is mentioned as I4.0 Language (I4.0L) that is specified in VDI/VDE 2193. It defines a set of rules compromising the vocabulary, the message structure and the semantic interaction protocols to organize the message sequences to the meaningful dialogues. I4.0L implements the bidding protocol based on the contract network protocol [29].

2.4 Agent Framework

Agent Frameworks ease development and the operation of large-scale, distributed applications and services. For us, it is especially important that the framework is open, modular, extensible and fosters the holonic concept. [21] and [24] list and discuss a number of agent platforms, while the work of [5] evaluates five different agent languages and frameworks. The results imply that especially SARL language running on Janus platform is superior to other systems in aspects concerning the communication between agents, modularity and extensibility. The biggest advantage to other languages is that there are no restrictions regarding the interactions between agents. These positive effects are balanced by the fact that debugging is limited to local applications and, above all, the transfer between the design to the implementation is very complicated. SARL supports dynamic creation of agent hierarchies and implements holonic architecture patterns. Finally, SARL is chosen to implement our MAS.

In [27], the authors explain that SARL Agents represent Holons. In the following, we prefer using the notation of a Holon. SARL uses the concepts of Behaviors and Holon Skills to define the entities. As explained in [27], a "[. . .] Behavior maps a collection of perceptions [. . .] to a sequence of Actions", whereas a Holon Skill implements a specification of a collection of Actions to make some change in the environment. A Capacity provides an interface for the collection of all actions. Though, the focus lies on reusability and lowering of complexity. To avoid confusion with the different meanings of the term skill, the following explicitly refers to Holon Skill or Production Skill. Through the usage of Holon Capacities and Skills, SARL follows modularization in analogy to Capabilities and Production Skill of resources.

3 Towards an Architecture for Modern Industrial Manufacturing Systems

The challenge for SP is represented in a need for an architecture that enables flexibility, customizability and copes with dynamic optimization and decision-making. The information model of Capabilities-Skills-Services of the *Plattform Industrie 4.0* [25] promises standardization to foster shop-floor-level reconfiguration and dynamic planning. Hence, AAS proves strength regarding the interoperability of system elements. The work of [14] demonstrates a way to use AAS to (re)plan and execute the production and make production resources interchangeable. The authors use AAS as Digital Twin for assets like products and resources. MAS controls the production modules of a work center. This interconnection follows the idea of encapsulation and modularity to enable flexible and technology-independent access to production resources. Encapsulation intends to increase reconfigurability at the shop-floor level. Resources should not perform rigid operations but be assigned tasks that they can process themselves. In [15], an SP network is presented that

enables data exchange in a self-sovereign interoperable manner. We implement that by combining concepts like Gaia-X, AAS and I4.0L. These technologies seem promising to enable a cross-company supply chain. In [15], we focus on the business layer communication to share data and explain the necessary components to build a data ecosystem. Nevertheless, there needs to be a component that enables logic to connect the shop floor to the connected world. Thus, [15] is our basis to realize inter-organizational communication and the following focuses on the intra-organizational resilience of the shop-floor.

3.1 Multi-agent System Manufacturing Architecture

Holonic MAS seems a promising pattern to wrap the factory's granularity and build complex and resilient systems. It is important to note that the technology to communicate with a customer and other factories might change over time or might differ for individual customers or factories. Consequently, to be technology-independent, our MAS does not include an explicit connection technology. MAS accumulates some of its resource capabilities into services and provides these services to the external world following the principles of service-oriented systems [9]. Besides, the holonic MAS supervises a production system to plan and execute the production and connects a factory to an SP network. Inspired by the upper mentioned concepts, a modern factory is represented in Fig. 3.

With reference to the described aspects, the system consists of three main Holons as displayed in Fig. 3. The basic structure follows the general idea of PROSA as described in Sect. 2. The difference between the presented MAS and their architecture is three-fold. First, the tasks of the management of the products are fulfilled by AAS instead of having a Product Holon in PROSA. A more detailed discussion about this change is discussed in 5. Second, the management of orders by

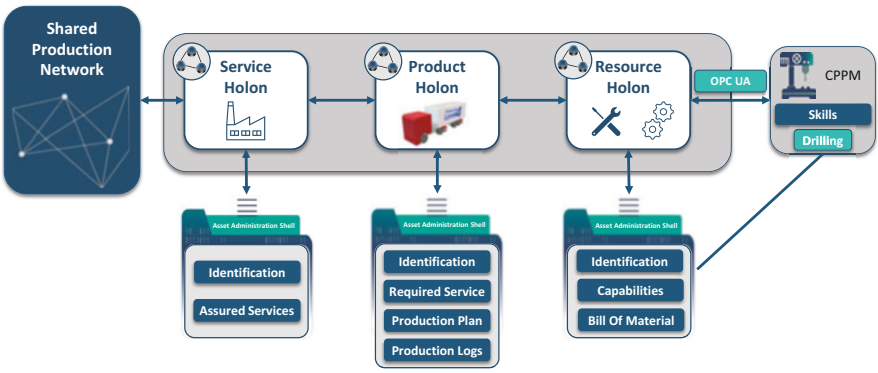

Fig. 3 The structure of the multi-agent system

the Order Holon is shifted from the Order Holon to the Product Holon. The reason for the new name is that our Product Holon takes care of orders and connects to Product AAS and thus encapsulates two tasks. Third, another Holon called Service Holon is added as an external representation of the factory layer as additional use case to PROSA which does not examine a SP scenario in detail and represents the factory as centralized HMS which spawns Order Holons based on the requests. However, we prefer to decouple the task of spawning Holons and the representation of the factory to achieve a higher resilience and flexibility.

Besides these changes, all three Holons are equipped with their own AAS to expose their self-description. This includes an SM Identification to identify the Holon in the MAS and an SM towards the interfaces of the Holon to be able to communicate with different communication technologies and to implement the communication technology independence. In addition, the SM Topology of each Holon describes the internal structure. Part of the structure are all aggregated Sub-Holons of the Holon. This SM eases the initialization of each Holon, especially in case of a restart.

In the following two subsections, we present details about the Service Holon in Sect. 3.2 and about the Product Holon in Sect. 3.3. For the Resource Holon, we will give a more detailed description in Sect. 4. The Resource Holon executes production steps by controlling and managing all resources. Each resource is via OPC UA connected to one Resource Holon, which controls its execution. For that purpose, the Resource Holon uses the Resource AAS to store information about the resource. This includes an SM Identification, the provided capabilities of the resource and the SM Bill of Material. In addition, the Resource Holon deals with tasks like lifecycle management of resources, human-machine interaction, resource monitoring and handling multiple communication technologies between the resources.

3.2 Service Holon

The Service Holon displayed on the left side of Fig. 3 manages and provides the services of the factory to the connected world. One example of a service is the 3D-printing service, which offers the capability of producing customized products using a fused deposition modeling process. Besides offering services, the Service Holon enables the MAS to process external tasks like ordering a specific product or executing a provided service. Besides that, the Service Holon takes care of the disposition of external services, products or substitutes. Therefore, the Service Holon has the ability to communicate with the SP network via an asynchronous event-based communication based on I4.0L. This implements VDI/VDE 2193 [34, 35] as the chosen communication standard. A more detailed use case for the connection to SP networks is applied in [15].

In the context of external SP Networks, the Service Holon represents the interface to the factory. For this reason, the Service Holon takes care of the Factory AAS. This AAS uses a unique identifier, a name and other general information to identify the

factory. Furthermore, it contains a description of all assured services, the factory has to offer. Based on this service catalog, other network members can request an offered service.

In case of incoming production requests, the Service Holon processes these requests and requests the Product Holon to handle the request. The communication to the Product Holon is like the overall communication between all Holons in the present architecture, asynchronous and event-based.

3.3 Product Holon

The Product Holon takes care of the production process and subdivides production tasks into different subtasks. In this context, the holonic approach takes effect. Each Holon is responsible for one task and spawns for every subtask a Sub-Holon. Together, they display the production process in a tree-like manner. To handle these incoming tasks or derived subtasks, each Holon triggers an internal execution at the Resource Holon or requests an external execution from the Service Holon.

In the case of an internal execution, each Holon needs to check if the execution is feasible. Therefore, the Product Holon first matches the capabilities given by the resources to the desired capability to fulfill the (sub-)task. If both capabilities fit together, a feasibility check on the resources is triggered to simulate if a resource is able to perform the task under posed conditions (e.g., if the process can supply the desired product quality and evaluate estimated time, costs and consumption). After a successful feasibility check, the Product Holon spawns an AAS as Digital Twin for the product. The Product AAS contains information towards the product identification like a name and a unique identifier. If the AAS contains some subtasks, which require an external execution, the Product AAS contains a description of all required external services to execute the subtask. After starting the production process, the Product Holon further controls the process by triggering production steps or monitoring the current production state. To monitor the process, the Product Holon updates the corresponding Product AAS by adding logging data to the production log.

To illustrate the execution of the Product Holon, a model truck as sample product is ordered via the Service Holon. The truck is assembled out of two different semitrailers. The semitrailer_truck consists of a 3D-printed cabin pressed on a brick chassis called cab_chassis. Similar to the semitrailer_truck, the other semitrailer is built by mounting a 3D-printed or milled trailer onto a semitrailer_chassis. Figure 4 shows the corresponding product structure tree.

Each of the displayed components and component assemblies relates to a production step to produce the respective component. First, the components need to be manufactured, then the semitrailers are assembled from the components and at the end, the trailer is mounted on the cab_chassis to assemble the full truck. For each of the given components, an own Product AAS is spawned. For example, the truck Holon spawns on the highest level of the Bill of Material two semitrailer Holons.

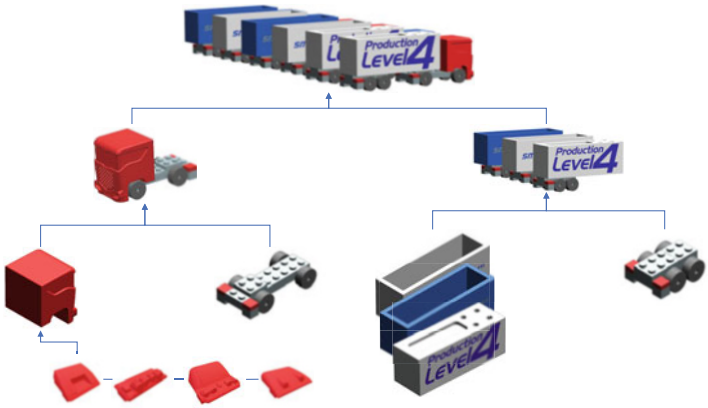

Fig. 4 The product tree of the model truck

Both Holons independently produce their related semitrailers and after completion, they report back to the truck Holon, which then controls the production of the full truck by controlling the assembly step of both semitrailers.

4 Execution System: Resource Holon

The Resource Holon takes care of the management of CPPMs. As visualized in Fig. 1, the Resource Holon serves as a proactive entity using the APIs of OPC UA as well as AAS and enables dynamic interactions. All AASs are deployed using the BaSyx middleware (v1.3) [18]. BaSyx is an open-source tool targeting to implement the specification of the AAS [10, 11] and provides additional services like storing AAS in databases, authorization and notifying the user with data change events. Holons collaboration fosters flexible execution and planning, while AAS and Production Skills provide interoperability. Next to these aspects, we want to emphasize resilience on the software layer. Resource Holons are deployed as Docker containers and managed by a GitLab repository that automatically creates container images and allows continuous deployment. Modern industrial environments need applications that are isolated and separated from the runtime environment to switch the underlying hardware and balance the load.

Our Resource Holon consists of an arbitrary number of Holons, where we distinguish between a Resource Holon of type Island and CPPM. The former is used to emphasize the existence of Sub-Holons, while the latter type is used to highlight the smallest possible entity that cannot be further partitioned. For interactions, it is not significant whether the Sub-Holon is an Island or a CPPM. We use this differentiation to separate the modules to build the Holons, i.e., to classify the Behaviors and Holon Skills for each type. Island Resource Holons are more responsible for lifecycle management and coordination, while CPPM Resource

Fig. 5 The behaviors and skills of an Island resource Holon

Holons schedule and perform the concrete tasks. Furthermore, Island Resource Holons encapsulate the sum of all Sub-Holons, by providing proxy functionality.

4.1 Behaviors and Skills of a Holon

SARL Holons consist of a collection of Behaviors and Holon Skills. Island Resource Holons provide hierarchies and coordinate Sub-Holons, whereas CPPM Holons, as the smallest entity, are connected to physical assets. Figure 5 summarizes Island Resource Holon's Behaviors and Skills. Each Island Resource Holon has an AAS that describes the Holon's properties, configures and parametrize the system. The key aspects of AAS are to provide information on how to find a Holon, how a Holon is constructed, and how the Holon's interfaces are defined. This information is available in the SM Topology and Interface (see Sect. 3.1) and is also provided to Sub-Holons. The AAS Skill extracts the Holons' information as well as the environmental information. The Update Behavior is used to gather information about the contained Holons' states, update the topology when MAS changes, and synchronize this information with AAS. Since the Island Resource Holon manages Sub-Holons, the Lifecycle Skill allows to dynamically create, destroy or reboot Holons in its own holonic context. As SARL Janus provides message channels for all Holons in runtime, the communication with other Holons requires an external communication interface. The Inter-Holon Behavior allows the external communication to Holons in other runtimes, e.g., message exchange between the Resource Holon and the Product Holon via the open standard communication middleware Apache Kafka. For communication and understanding, an I4.0 Message Skill supports accordance with a standardized message model according to the

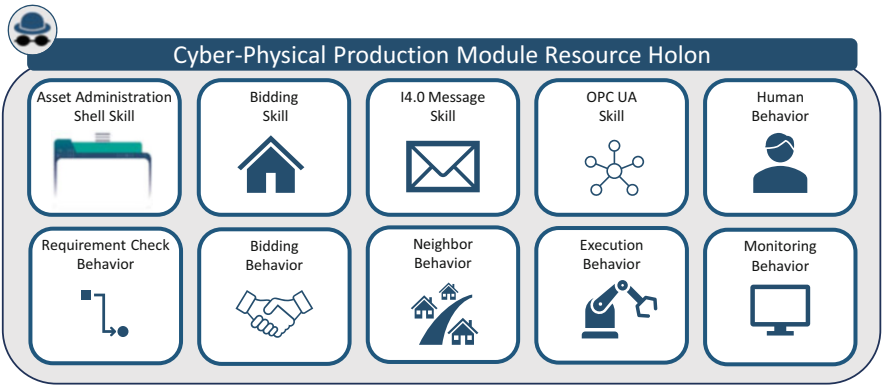

Fig. 6 The behaviors and skills of a CPPM resource Holon

VDI/VDE 2193-1 [35]. Communication, collaboration and negotiation are the key components for a successful process. Island Resource Holon responds to production requests of the Product Holon, initialize negotiations and sends production requests to Sub-Holons. In Negotiation Behavior, the Island Resource Holon verifies incoming messages. Depending on the request, Island Resource Holon forces Sub-Holons to follow a task or request the possible execution. The former is used when static optimization is applied, i.e., a global schedule shall be executed. An example of global scheduling is demonstrated in [13], in which we schedule value-adding as well as non-value-adding processes. The latter implements the bidding protocol to foster dynamic optimization. Negotiation behavior defines the duration of auctions and chooses incoming offers o based on max operator, i.e., for n incoming offers, the chosen offer calculates with $o_i = \max(o_1, \ldots, o_n)$. Next to software systems, Holons may interact with humans, intending a special treatment in terms of prioritization. Therefore, Human Behavior considers human knowledge and adjustments. We are not trying to accomplish fully automated plants and exclude humans from production. Instead, we want to support human decisions to benefit from experiences and intuition, as well as building factories for humans [36].

For CPPM Resource Holons some building blocks like the Asset Administration Shell Skill, the I4.0 Message Skill and the Human Behavior overlap (see Fig. 6). For interactions, the CPPM Resource Holon provides three Behaviors: Requirement Check, Bidding and Neighbor. Requirement Check acts in loose adherence to a method call to achieve control structures in a hierarchy. In case, Island Resource Holon fosters execution, the CPPM Resource Holon verifies if it can follow the call and starts or queues the task. In Bidding Behavior, the Bidding Skill is used to calculate a bid in the range between 0 and 1. The bid determines the desire to perform the job. This fosters dynamic optimization, while taking processing time, changeover times, deadlines, availability and resource's possible operations into account. The calculation process is achieved using a Reinforcement Learning algorithm. The basis of the Reinforcement Learning algorithm is described in [26], while

a modified variant will be published in future work. The last interaction pattern is the neighbor behavior. CPPMs sense their environment, thus, CPPM Resource Holon can omit hierarchies and directly communicate with their physical neighbor to perform a complex task in a collaborative way. An example is a Pick&Place operation that usually requires the supply by transportation means. The CPPM Resource Holon has additional functions regarding the control and monitoring of Production Skills. The Execution Behavior builds an event-based sequence to reliably execute a Production Skill. In this context, it represents pipelines to set the Production Skill's parameters, verify compliance of all preconditions, tracks the execution state and manages postconditions. Therefore, the CPPM Resource Holon uses the OPC UA Skill, which allows access to a Production Skill interface directly deployed on the CPPM. Furthermore, CPPM Resource Holon has a Monitoring Behavior, which is used to check relevant sensor data, track the system status and update system-critical information. In the future, anomaly detection and supervision will also be implemented in this behavior.

4.2 Demonstrator Use Case

To demonstrate the application of the Resource Holon in a production environment, *Produktionsinsel_KUBA* of a real-world factory at *SmartFactory*^{KL} is used (see Fig. 7).

Produktionsinsel_KUBA consists of three CPPMs named Connector Module, Quality Control Module and Conveyor Module as well as a *Produktionsinsel_SYLT*

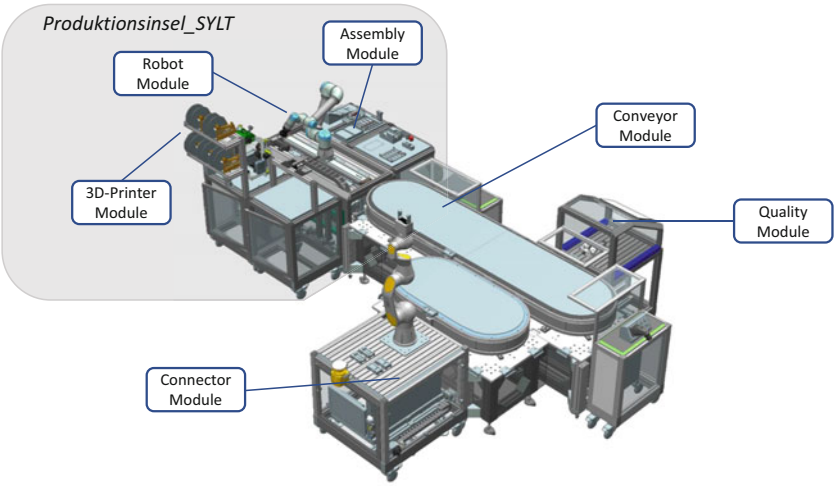

Fig. 7 *SmartFactory*^{KL} real-world demonstration factory: *Produktionsinsel_KUBA*

enclosing a 3D Printer, a Robot and a Hand Assembly. The Connector Module serves as a supply and storage station for components parts. The Connector Module transfers the delivered components onto the Conveyor Module. The Conveyor Module transports the components to the *Produktionsinsel_SYLT*, where individual parts are mounted into a higher order assembly. Afterward, the quality of the product is checked, and the assembly is ejected at the Connector Module. Our exemplary production assembles a model truck shown in Fig. 4. In this context, *Produktionsinsel_KUBA* is incapable of producing all the components of the model truck on its own. Therefore, our scenario assumes that the required components of the model truck have already been manufactured in the sense of SP as explained in [15] and delivered to the Connector Module. The CPPMs can be positioned in an arbitrary layout. As a result, *Produktionsinsel_KUBA* offers different services depending on connected modules and increases flexibility. To provide a safe workspace, the CPPMs are mechanical locked. In this context, we refer to two or more mechanical connected CPPMs as neighbors. In [12], we present the development of our Conveyor Module to enable on-demand transportation. We mention five different coupling points on which CPPMs can be locked. The mechanism is realized with magnets and RFID sensors. Hence, the Connector Module, the Quality Control Module and the *Produktionsinsel_SYLT* are physically locked to the Conveyor Module, building a connected neighborhood. The RFID tag contains the CPPM's ID that allows to identify the locked module. Therefore, the self-description is accessible and CPPM Resource Holons can build peer-to-peer connections. Our production process starts when the Resource Holons gets the request to execute a production step from the Product Holon. The production step is described in a production plan following the metamodel of AAS. An example of a production plan is visualized in [14].

Based on the idea to encapsulate the manufacturing logic of production modules in Resource Holons, the interaction between the Island Resource Holon and the responding Sub-Holons is described. Our instantiated *Produktionsinsel_KUBA* Holon is visualized in Fig. 8. During the request to execute a production step, the *Produktionsinsel_KUBA* Resource Holon verifies the query in Negotiation Behavior and asks the Sub-Holons if they can perform the required effect. Since we omit global scheduling, dynamic optimization through collaboration takes place. The Sub-Holons compete to perform the effect while calculating a bid using the Bidding Skill. If the underlying CPPM is incapable of performing the action, the bid results in 0. Otherwise, the bid is calculated through Reinforcement Learning methods with a maximum value of 1. The *Produktionsinsel_KUBA* Resource Holon verifies the bids, by ignoring all offers $o < 0.2$ and distributes the steps. The CPPM Resource Holons' local decision leads to a global behavior, where the global allocation of resources is optimized by the bidding system. This procedure ensures local CPPM Resource Holon utilization and realizes on-demand scheduling. To perform a required effect, the CPPM Holons access Production Skills via the Execution Behavior to perform a change in the environment. During the execution of a task, Holons may cooperate to perform tasks they are unable to do on their own. As an example of the communication between different Holons, the Connector Holon

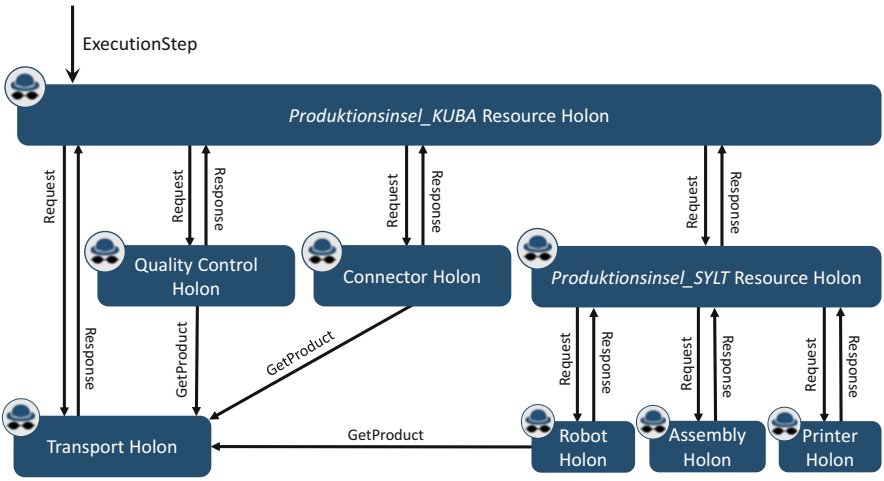

Fig. 8 Instantiation of *Produktionsinsel_KUBA* resource Holon

and the *Produktionsinsel_SYLT* Resource Holon communicate with the Conveyor Holon to order transportation means for a specified product. Conveyor Holon manages the orchestration of the transportation means by routing and queuing. This communication relies more on a call, since Conveyor Holon encapsulates on-demand transport.

This sequence describes the demonstration scenario at *Produktionsinsel_KUBA*. The combination of requesting and calling Holon's abilities leads to flexible control structures that allow manufacturing in a resilience way. As a result, we can handle static and dynamic requirements. Additionally, the modular structure in combination with usage of AAS and Production Skills foster interoperability and interchangeability at different layers of the factory, from the shop-floor to the connected world.

5 Conclusion

This chapter presents a MAS approach in the manufacturing domain, which enables a factory to control its resources, to define and manage products and to provide services to other SP participants. The MAS is based on a holonic approach and is subdivided into Holons, each taking care of one of these tasks. For us, a Holon is treated as a special type of Agent with additional characteristics regarding recursiveness and connection to the hardware. The MAS collaborates with and uses modern Industry 4.0 technologies such as Production Skills, AAS or OPC UA.

The presented MAS is enrolled on a demonstrator testbed at *SmartFactory*^{KL}, which is part of an SP scenario to produce model trucks. We divide our manufac-

turing system into three Holons. The Service Holon provides and retrieves services from the connected world. The Product Holon deals with modular encapsulated products to manage dependencies between individual parts and assemblies as well as controlling the production process. The Resource Holon encapsulates the layer of the production testbed and connects the virtual with the physical world. To guarantee autonomy, our Resource Holons use descriptions of AAS to gain knowledge about the environment and use CPPM's Production Skills to perform an effect in the physical world. We achieve a flexible and resilient system by providing communication patterns that allow hierarchical and heterarchical modularization.

However, the current state of our MAS is subject to different limitations. This means that it will be extended in the future to fulfill different other features and will solve different topics. One topic is to put more emphasize on product's lifecycle, while providing more complex planning systems to extract product's features, match capabilities and trace tender criteria. Another extension is planned on the monitoring system to embed a factory wide monitoring system to combine a supervision of the production process, factory level information like assured services and resource data. The last topic is to provide more generalized holonic patterns and give more insights about the Service Holon and the Product Holon.

As a result, we want to compare our architecture to other MAS systems, with a special focus on the applied technologies in our systems. One of these technologies is AAS. In comparison to full agent-based solutions, we typically replace one Holon (e.g., in PROSA the Product Holon). As a downside, this leads to more applied technologies in the system (due to the different technology stack) and thus to a more complex architecture. However, the AAS as manufacturing standard supports the interoperability between other factories and a simple data exchange format. Furthermore, we use one standardized data format to express all our knowledge to ease the internal usage of data via a system-wide interface.

Besides AAS as data format, we use SARL as Agent Framework. SARL itself is a domain-specific language, which leads to a couple of general advantages and disadvantages, as explained in [6]. We want to take up some of the listed problems and advantages and add a few more SARL language-specific arguments. First, SARL is especially designed to build MAS and includes an own metamodel to define the structure of a Holon. Besides that, SARL offers concepts to encapsulate certain functionality in Behaviors and Skills and leads to a modular system. One special feature of SARL is that Holons are able to control the lifecycle of other Holons, which is quite close to our applied concept of the MAS. Although SARL is functional suitable for us, SARL also has different disadvantages. For example, it is hard to find a documentation and help in the community if SARL specific problems occur. Unfortunately, developing SARL code is exhausting since general supported development environments do not always react in our desired response time.

Another difference between our MAS concept and other MAS concepts is the granularity of applied Holons. In many cases, each device (e.g., a robot or even in a smaller granularity like a sensor) has an own Holon. In our approach, a Holon connects to one CPPM, which encapsulates single resources like robot arms or 3D-printers. In this approach, a Holon accesses each CPPM by calling

their provided skills. Having Holons on the device level leads to more holonic communication, and thus more resources and effort is required to handle the holonic communication. Moreover, MAS does not need to operate in real-time to perform actions without a delay. This is why, we decided to encapsulate internal communication inside a CPPM and keep time-critical and safety-critical tasks in the physical processing parts. Furthermore, Holons are independent of machine-specific control technologies, which increase the flexibility of the system towards resource-specific technologies. Finally, we want to mention that even for small holonic MAS, communication quickly becomes complex and lacks transparency. Using standardized technologies like OPC UA and AAS regains this transparency and supports application in the factory.

Acknowledgments This work has been supported by the European Union's Horizon 2020 research and innovation program under the grant agreement No 957204, the project MAS4AI (Multi-agent Systems for Pervasive Artificial Intelligence for Assisting Humans in Modular Production Environments).

References

1. Beheshti, R., Barmaki, R., Mozayani, N.: Negotiations in holonic multi-agent systems. Recent Advances in Agent-Based Complex Automated Negotiation pp. 107–118 (2016). https://doi.org/10.1007/978-3-319-30307-9_7
2. Bergweiler, S., Hamm, S., Hermann, J., Plociennik, C., Ruskowski, M., Wagner, A.: Production Level 4" Der Weg zur zukunftssicheren und verlässlichen Produktion (2022), https://smartfactory.de/wp-content/uploads/2022/05/SF_Whitepaper-Production-Level-4_WEB.pdf, (Visited on 18.05.2023)
3. Braud, A., Fromentoux, G., Radier, B., Le Grand, O.: The road to European digital sovereignty with Gaia-X and IDSA. IEEE network **35**(2), 4–5 (2021). https://doi.org/10.1109/MNET.2021.9387709
4. Christensen, J.: Holonic Manufacturing Systems: Initial Architecture and Standards Directions. In: First European Conference on Holonic Manufacturing Systems, Hannover, Germany, 1 December 1994 (1994)
5. Feraud, M., Galland, S.: First comparison of SARL to other agent-programming languages and frameworks. Procedia Computer Science **109**, 1080–1085 (2017). https://doi.org/10.1016/j.procs.2017.05.389
6. Fowler, M.: Domain-specific languages. Pearson Education (2010)
7. Giret, A., Botti, V.: Holons and agents. Journal of Intelligent Manufacturing **15**(5), 645–659 (2004). https://doi.org/10.1023/B:JIMS.0000037714.56201.a3
8. Hermann, J., Rübel, P., Birtel, M., Mohr, F., Wagner, A., Ruskowski, M.: Self-description of cyber-physical production modules for a product-driven manufacturing system. Procedia manufacturing **38**, 291–298 (2019). https://doi.org/10.1016/j.promfg.2020.01.038
9. Huhns, M.N., Singh, M.P.: Service-oriented computing: Key concepts and principles. IEEE Internet computing **9**(1), 75–81 (2005). https://doi.org/10.1109/MIC.2005.21
10. Industrial Digital Twin Association: Specification of the Asset Administration Shell Part 1: Metamodel, https://industrialdigitaltwin.org/wp-content/uploads/2023/06/IDTA-01001-3-0_SpecificationAssetAdministrationShell_Part1_Metamodel.pdf, (Visited on 28.06.2023)
11. Industrial Digital Twin Association: Specification of the Asset Administration Shell Part 2: Application Programming Interface, https://industrialdigitaltwin.org/wp-content/uploads/

2023/06/IDTA-01002-3-0_SpecificationAssetAdministrationShell_Part2_API_.pdf, (Visited on 28.06.2023)
12. Jungbluth, S., Barth, T., Nußbaum, J., Hermann, J., Ruskowski, M.: Developing a skill-based flexible transport system using OPC UA. at-Automatisierungstechnik **71**(2), 163–175 (2023). https://doi.org/10.1515/auto-2022-0115
13. Jungbluth, S., Gafur, N., Popper, J., Yfantis, V., Ruskowski, M.: Reinforcement Learning-based Scheduling of a Job-Shop Process with Distributedly Controlled Robotic Manipulators for Transport Operations. IFAC-PapersOnLine **55**(2), 156–162 (2022). https://doi.org/10.1016/j.ifacol.2022.04.186, https://www.sciencedirect.com/science/article/pii/S2405896322001872, 14th IFAC Workshop on Intelligent Manufacturing Systems IMS 2022
14. Jungbluth, S., Hermann, J., Motsch, W., Pourjafarian, M., Sidorenko, A., Volkmann, M., Zoltner, K., Plociennik, C., Ruskowski, M.: Dynamic Replanning Using Multi-agent Systems and Asset Administration Shells. In: 2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA). pp. 1–8 (2022). https://doi.org/10.1109/ETFA52439.2022.9921716
15. Jungbluth, S., Witton, A., Hermann, J., Ruskowski, M.: Architecture for Shared Production Leveraging Asset Administration Shell and Gaia-X (2023), unpublished
16. Koestler, A.: The ghost in the machine. Macmillan (1968)
17. Kolberg, D., Hermann, J., Mohr, F., Bertelsmeier, F., Engler, F., Franken, R., Kiradjiev, P., Pfeifer, M., Richter, D., Salleem, M., et al.: SmartFactoryKL System Architecture for Industrie 4.0 Production Plants. SmartFactoryKL, Whitepaper SF-1.2 **4** (2018)
18. Kuhn, T., Schnicke, F.: BaSyx, https://wiki.eclipse.org/BaSyx, (Visited on 07.07.2022)
19. Leitão, P., Colombo, A.W., Restivo, F.J.: ADACOR: A collaborative production automation and control architecture. IEEE Intelligent Systems **20**(1), 58–66 (2005). https://doi.org/10.1109/MIS.2005.2
20. Leitao, P., Karnouskos, S. (eds.): Industrial Agents: Emerging Applications of Software Agents in Industry. Kaufmann, Morgan, Boston (2015). https://doi.org/10.1016/C2013-0-15269-5
21. Leon, F., Paprzycki, M., Ganzha, M.: A review of agent platforms. Multi-paradigm Modelling for Cyber-Physical Systems (MPM4CPS), ICT COST Action IC1404 pp. 1–15 (2015)
22. Liu, Y., Wang, L., Wang, X.V., Xu, X., Jiang, P.: Cloud manufacturing: key issues and future perspectives. International Journal of Computer Integrated Manufacturing **32**(9), 858–874 (2019). https://doi.org/10.1080/0951192X.2019.1639217
23. Neubauer, M., Reiff, C., Walker, M., Oechsle, S., Lechler, A., Verl, A.: Cloud-based evaluation platform for software-defined manufacturing: Cloud-basierte Evaluierungsplattform für Software-defined Manufacturing. at-Automatisierungstechnik **71**(5), 351–363 (2023). https://doi.org/10.1515/auto-2022-0137
24. Pal, C.V., Leon, F., Paprzycki, M., Ganzha, M.: A review of platforms for the development of agent systems. arXiv preprint arXiv:2007.08961 (2020). https://doi.org/10.48550/arXiv.2007.08961
25. Plattform Industrie 4.0: Information Model for Capabilities, Skills & Services, https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/CapabilitiesSkillsServices.html, (Visited on 06.06.2023)
26. Popper, J., Ruskowski, M.: Using Multi-agent Deep Reinforcement Learning for Flexible Job Shop Scheduling Problems. Procedia CIRP **112**, 63–67 (2022). https://doi.org/10.1016/j.procir.2022.09.039
27. Rodriguez, S., Gaud, N., Galland, S.: SARL: a general-purpose agent-oriented programming language. In: 2014 IEEE/WIC/ACM International Joint Conferences on Web Intelligence (WI) and Intelligent Agent Technologies (IAT). vol. 3, pp. 103–110. IEEE (2014). https://doi.org/10.1109/WI-IAT.2014.156
28. Ruskowski, M., Herget, A., Hermann, J., Motsch, W., Pahlevannejad, P., Sidorenko, A., Bergweiler, S., David, A., Plociennik, C., Popper, J., et al.: Production bots für production level 4: Skill-basierte systeme für die produktion der zukunft. atp magazin **62**(9), 62–71 (2020). https://doi.org/10.17560/atp.v62i9.2505

29. Smith, R.G.: The contract net protocol: High-level communication and control in a distributed problem solver. IEEE Transactions on computers **29**(12), 1104–1113 (1980). https://doi.org/10.1109/TC.1980.1675516
30. Trunzer, E., Calà, A., Leitão, P., Gepp, M., Kinghorst, J., Lüder, A., Schauerte, H., Reifferscheid, M., Vogel-Heuser, B.: System architectures for Industrie 4.0 applications: Derivation of a generic architecture proposal. Production Engineering **13**, 247–257 (2019). https://doi.org/10.1007/s11740-019-00902-6
31. Valckenaers, P.: Perspective on holonic manufacturing systems: PROSA becomes ARTI. Computers in Industry **120**, 103226 (2020). https://doi.org/10.1016/j.compind.2020.103226
32. Van Brussel, H., Wyns, J., Valckenaers, P., Bongaerts, L., Peeters, P.: Reference architecture for holonic manufacturing systems: PROSA. Computers in Industry **37**(3), 255–274 (1998). https://doi.org/10.1016/S0166-3615(98)00102-X, https://www.sciencedirect.com/science/article/pii/S016636159800102X
33. Van Leeuwen, E., Norrie, D.: Holons and holarchies. Manufacturing Engineer **76**(2), 86–88 (1997). https://doi.org/10.1049/me:19970203
34. VDI/VDE-Gesellschaft Mess- und Automatisierungstechnik: Language for I4.0 components - Interaction protocol for bidding procedures, https://www.vdi.de/en/home/vdi-standards/details/vdivde-2193-blatt-2-language-for-i40-components-interaction-protocol-for-bidding-procedures, (Visited on 01.06.2023)
35. VDI/VDE-Gesellschaft Mess- und Automatisierungstechnik: Language for I4.0 Components - Structure of messages, https://www.vdi.de/en/home/vdi-standards/details/vdivde-2193-blatt-1-language-for-i40-components-structure-of-messages, (Visited on 01.06.2023)
36. Zuehlke, D.: SmartFactory–Towards a factory-of-things. Annual reviews in control **34**(1), 129–138 (2010). https://doi.org/10.1016/j.arcontrol.2010.02.008

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.